Feasibility Analysis

MIMIC-IV QA Benchmark for Coverage vs. Dispersion

Contents

1 Introduction

The experiment's goal is that *coverage* (facility-location) outperforms *dispersion* (MMR, gMMR, FPS) when the candidate pool of retrieved chunks has a multi-cluster structure. Other QA datasets exist (HotpotQA, 2WikiMultiHopQA, MuSiQue), but these benchmarks have fixed candidate pools that we cannot control.

Ideal Cluster Structure for coverage

1. **Redundant central cluster** - dominant topic, many chunks VERY similar to one another.
2. **Complementary cluster(s)** - other relevant aspects of the query but of less similar points. Coverage ensures at least one representative from each.
3. **Niche cluster** - small but informative, not an outlier.
4. **Outliers** - informative or noisy. Dispersion favors them (maximizes spread); coverage ignores them unless they represent a dense region.

Proposal: build a **custom QA benchmark from MIMIC-IV** where we can (a) verify that candidate pools have the desired cluster structure, (b) control pool composition, and (c) design multi-aspect queries that require many sources.

Part I

MIMIC-IV Dataset

The database is split across two datasets (important CSVs highlighed in blue):

- **MIMIC-IV** ⇒ tabular clinical data recording every diagnosis, procedure, and medication for each hospitalization. This can give us **ground truth** for evaluation. There are two submodules:
  - **hosp**: patient tracking (`patients`, `admissions`, `transfers`), diagnoses (`diagnoses_icd`, `d_icd_diagnoses`, `procedures_icd`, `drgcodes`), labs (`labevents`), medications (`prescriptions`, `emar`).
  - **icu**: bedside data (`chartevents`, `inputevents`, `outputevents`, `procedureevents`).
- **MIMIC-IV-Note** ⇒ free-text clinical notes
  - `discharge`: discharge summaries. Long, narrative clinical documents written by doctors when a patient leaves the hospital.
  - `radiology`: radiology reports.

For our purposes, the critical tables are:

- `discharge` (in MIMIC-IV-Note): ∼331K discharge summaries.
- `diagnoses_icd`: join table linking ICD-9/ICD-10 (International Classification of Diseases) codes to subject hospitalizations, with approximate importance ordering (`seq_num`). A typical admission has 5-20 ICD codes, often spanning multiple body systems. Definitions for the codes themselves are in the `d_icd_diagnoses` table
- `patients`: gender, `anchor_age`, date of death.
- `admissions`: admission type, insurance, language, marital status, race, discharge location.

All tables link via `subject_id` (patient) and `hadm_id` (hospitalization). The dataset has 331,793 discharge summaries from 145,914 patients. All protected health information (names, dates, locations) has been removed or replaced with placeholders (___).

2 MIMIC-IV-Note

The notes are of two kinds: **Discharge Summaries** and **Radiology Reports**.

2.1 Discharge Summaries

The structure of the notes seems consistent, with standardized sections:

Section	Content	QA Value
Brief Hospital Course	**Clinical reasoning per problem**	**Highest**
History of Present Illness	Symptom narrative, temporal progression	High
Pertinent Results	Labs, imaging, micro interpretations	High
Discharge Diagnosis	Diagnostic labels	High (linkable to ICD)
Chief Complaint	1–2 lines, reason for admission	**High** only as contextual embedding metadata
Past Medical History	Comorbidities list	Medium
Physical Exam	Admission + discharge findings	Medium
Discharge Medications	Treatment protocols with dosing	Medium
Major Surgical or Invasive Procedure	Procedures performed during stay	Low
Allergies	-	Low
Family History	-	Low
Medications on Admission	-	Low
Discharge Disposition	Destination (home, SNF, rehab, ...)	Low
Discharge Condition	Mental status, consciousness, activity	Low
Discharge Instructions	Patient-facing plain-language summary	Low
Service	Medicine, Surgery, ...	None
Attending	Attending physician	None (deidentified)
Social History	Lifestyle, substance use	None (deidentified)
Followup Instructions	Outpatient follow-up appointments	None (deidentified)

Brief Hospital Course. This section uses `#` markers to delimit individual clinical problems (e.g., `# Stroke`, `# Hypertension`, `# DVT`). Each sub-section contains *clinical reasoning*: differential diagnoses, treatment rationale, management decisions with justifications. This is where generalizable medical knowledge is embedded within patient-specific narratives.

However, the `#`-header format is not universal. There are two patterns:

1. **Complex medicine admissions** (the interesting cases): the Hospital Course is organized as a problem list with `#` headers, one block per active clinical issue. Example: a cirrhosis patient's Hospital Course has `# ASCITES`, `# HEPATIC ENCEPHALOPATHY`, `# HYPONATREMIA`, `# HIV`, `# COPD` (5–8 blocks, each 50–200 tokens). Sometimes there are additional sub-headers like `CHRONIC ISSUES` or `ACUTE/ACTIVE ISSUES` grouping the problems. The formatting is inconsistent: `# Ascites`, `# ASCITES`, `#NSTEMI` (with or without space after `#`, mixed casing).

2. **Simple or surgical admissions** (single-problem cases): the Hospital Course is a single narrative paragraph with *no* `#` headers. This includes orthopedic (hip fracture), urological (nephrectomy), and straightforward medicine cases (dysphagia workup). These are analogous to single-hop queries. There is only one information peak, so coverage should add nothing.

The `TRANSITIONAL ISSUES` block. This (sometimes misspelled as TRANISTIONAL ISSUES) appears at the end of many Hospital Courses and contains follow-up action items ("recheck electrolytes", "follow up with hepatology"), not clinical narratives. These are task-oriented and do not describe clinical problems.

Deidentification artifacts. The ___ placeholder is everywhere in dates, names, ages, and locations. Brief Hospital Course and Physical Exam can contain few of them but largely are usable. Ages are recoverable from `anchor_age` in the `patients` table.

2.2 Radiology Reports

The `radiology` table in MIMIC-IV-Note contains ∼2.3M radiologist reports spanning x-ray, CT, MRI, and ultrasound. Reports follow a structured format with sections: **Examination**, **Indication**, **Technique**, **Comparison**, **Findings**, and **Impression**. The associated `radiology_detail` table provides exam codes and exam names (e.g., `CT HEAD W/O CONTRAST`, `LIVER OR GALLBLADDER US`) that can be used for filtering.

Redundancy with discharge notes. The "Pertinent Results" section of discharge summaries often summarizes key imaging findings. Radiology reports may not add unique facets beyond what is already captured there.

I think Discharge Notes are better as a starting point and this can be used later if needed.

2.3 Considerations

2.3.1 Patient-specific vs. Generalizable data

The queries should ask about **clinical management patterns**: things that are described in discharge notes (the Brief Hospital Course says what was done and why):

> "*How is pneumonia managed in an elderly patient with chronic kidney failure?*"

This works because: (a) management decisions (antibiotic choice, fluid balance, renal dosing) are explicitly documented in discharge notes; (b) answering requires synthesizing across multiple patients: some with pneumonia alone, some with pneumonia + CKD, providing both the baseline and the modifier; (c) ground truth is derivable: chunks from admissions carrying both pneumonia and CKD ICD codes, plus chunks from pneumonia-only admissions for contrast. Each chunk contributes one patient's management; the gold set is the collection of chunks whose union covers all required facets.

3 MIMIC-IV Structured Data

3.1 Hosp Module

The **hosp** module captures hospital-wide data from admission to discharge. Key table groups (tables highlighted in blue are useful for us):

Patient tracking. Three tables describe demographics and movement:

- `patients`: gender, `anchor_age` (deidentified age), date of death. Ages are shifted but intervals are preserved, so age *at admission* is recoverable.
- `admissions`: admission/discharge times, admission type (emergency, elective, urgent), insurance, language, marital status, race, discharge location, in-hospital mortality flag.
- `transfers`: ward-level ADT (admission–discharge–transfer) records tracking physical location throughout the stay.

Billing and diagnoses.

- `diagnoses_icd`: up to 39 ICD-9-CM / ICD-10-CM codes per admission with approximate importance ordering (`seq_num`). Definitions in `d_icd_diagnoses`. This is our primary source for condition-level labels and query scaffolding.
- `procedures_icd`: billed procedures (ICD-9-PCS / ICD-10-PCS). Definitions in `d_icd_procedures`.
- `drgcodes`: Diagnosis Related Group codes (AP-DRG and MS-DRG) assigning an overall cost category to each hospitalization.
- `hcpcsevents`: HCPCS billing codes for provided services (e.g. mechanical ventilation, ICU care). Definitions in `d_hcpcs`.

3.2 ICU Module

The **icu** module (94,458 ICU stays, 65,366 unique patients) contains bedside data from the MetaVision clinical information system (iMDsoft), limited to patients admitted to an ICU. All events reference a `stay_id` (unique per ICU stay), defined in `icustays` with admission/discharge times and care unit names (e.g. MICU, SICU, CCU).

Suitability for embeddings. ICU data is predominantly **numerical time-series** (vital signs, infusion rates, urine output) rather than natural language. These signals are not directly embeddable with sentence transformers, and converting them to text (e.g., "heart rate 92 bpm at 14:30") would produce low-information chunks unlikely to match condition-level queries semantically. The ICU module is not a primary source for our corpus.

Part II

QA Dataset Generation Pipeline

4 Pipeline draft

4.1 Phases

Phase 1: Corpus Construction

1. **Select target conditions.** Query `diagnoses_icd` for ICD code groups with ≥ 200 admissions. Prefer conditions with high **ICD co-occurrence richness**: conditions whose admissions carry diverse comorbidity profiles (e.g., pneumonia + CKD, pneumonia + CHF, pneumonia + diabetes) are more likely to produce multi-aspect candidate pools than conditions with uniform ICD profiles.

2. **Build per-condition candidate pools.** For each condition, gather all discharge note chunks from admissions with that ICD code. This is the full candidate set D for facility-location.

3. **Chunk by section + sub-problem.**
   - Split on section headers
   - Within `Brief Hospital Course`, split on `#` problem markers

4. **Attach metadata to each chunk:**
   - `subject_id`, `hadm_id`, section name
   - All ICD codes for that admission

- Demographics: age group, gender, race, insurance
- Admission type, discharge disposition

5. **Deduplicate same-patient chunks.** The same patient can have multiple hospitalizations with nearly identical boilerplate sections, e.g., `Past Medical History`, `Allergies`, and `Family History` are often copied verbatim across admissions.

   Strategy: compute a content hash (e.g., SHA-256 of normalized text) per chunk and deduplicate by (`subject_id`, section, hash). Check also if it's worth considering *only* the most up to date note for a certain patient for a subset of repetitive sections (Past Medical History, etc.)

Phase 2: Embeddings Generation

Raw chunks are too patient-specific to match condition-level queries. Following the **Contextual Retrieval** approach [1], prepend a metadata prefix to each chunk before embedding:

```
[age_group] [gender] [race] patient admitted for [primary ICD description].
Chief complaint: [chief_complaint].
Comorbidities: [top ICD descriptions].
Section: [section_name].
```

Check out Section 5 for more details.

The specific contents to include are to be decided, based on the queries we want to build.

Phase 3: Query Generation

1. Pick a primary condition (e.g., stroke, ICD-10: I63.*)

2. Pick 1–2 **modifier axes** from metadata. Each axis resolves to a filter on `hadm_id`, which is the join key to the discharge notes already chunked in Phase 1. Since chunk metadata was pre-attached in Phase 1 step 4, query generation simply selects which modifier values to combine into the prompt template.

   - **Comorbidity** (prior MI, diabetes, renal failure): co-occurring ICD codes on the same admission. A single `hadm_id` carries 5–20 ICD rows in `diagnoses_icd`; "stroke + diabetes" is the set of `hadm_id`s that have both `I63.*` and `E11.*`. The human-readable description for the prompt comes from `d_icd_diagnoses`.
   - **Demographic** (elderly, women, specific race): requires joining two tables. `patients` provides `gender` and `anchor_age`; `admissions` provides `race`, `insurance`, `marital_status`, and `admission_type`. Both join to `diagnoses_icd` via `subject_id` and `hadm_id` respectively.
   - **Complication** (post-surgical, recurrent, hemorrhagic transformation): draws from multiple sources. Co-occurring ICD codes capture complications (e.g., stroke `I63.*` + hemorrhage `I61.*` on the same `hadm_id`). `procedures_icd` identifies post-surgical modifiers. Recurrence is detected by finding the same `subject_id` with the target condition across multiple `hadm_id`s.

3. Prompt an LLM:

   > *"Generate a clinical management question about [condition] that requires knowing how treatment or workup changes when [modifier] is present. The answer should require synthesizing management decisions from multiple patient cases, not textbook symptom lists."*

4. **Divergence pre-filter.** Before investing in expensive gold annotation, run facility-location and top-k on the candidate pool for the generated query and compute two cheap signals:

   - **Jaccard divergence**: if $J(S_{\text{cov}}, S_{\text{topk}}) \approx 1$, coverage selects nearly the same chunks as top-k $\Rightarrow$ skip this query, coverage adds nothing here. If $J \ll 1$, coverage diverges $\Rightarrow$ proceed.

- **Facility-location gap**: $g(S_{\text{cov}}) - g(S_{\text{topk}})$ quantifies how much pool structure top-k ignores. Both S_{cov} and S_{topk} are already computed, so this is free.

Only queries where coverage meaningfully diverges from top-k proceed to gold annotation.

5. **Gold answer generation**

The idea: give an LLM access to the candidate pool, have it answer the query comprehensively, and for each piece of information in the answer **cite which chunk(s)** it drew from. The cited chunks become the **gold set**, each tagged with a facet label.

Facet annotation. The gold answer naturally covers several distinct information aspects — each one is a **facet**. Facet labels are not pre-defined; the LLM generates them during annotation. For each piece of information in its answer, the LLM cites which chunk(s) it drew from and assigns a short descriptive label for the aspect of care it addresses.

Example. For the query *"How does management of acute stroke differ when the patient has concurrent CKD?"*, the annotator might produce:

Facet label	What it captures	Example chunk content
`anticoag_adjustment`	CKD affects blood thinner dosing	Heparin dose reduced to 10u/kg/hr due to CrCl < 30
`contrast_avoidance`	CKD contraindicates contrast imaging	CT angiography deferred, nephropathy risk
`bp_target`	Different BP targets in stroke + CKD	Permissive HTN limit adjusted for renal function
`dialysis_timing`	When to initiate dialysis during acute stroke	Nephrology consulted for HD timing peri-stroke

Multiple chunks can map to the same facet (e.g., two different patients' notes both discuss anticoagulation adjustment). Aspect recall does not require retrieving *all* chunks for a facet — just *at least one* per facet (the $\exists$ in the AR formula, Section 4.1).

Exhaustive tagging and strict evaluation. The map pass reads *every* chunk in the pool, so the LLM is instructed to tag **all** chunks that support each facet, not just the single best one. This makes the gold set comprehensive. At evaluation time, aspect recall uses **strict chunk-id matching**: a retrieved chunk contributes to a facet only if it appears in the gold set for that facet.

Gold annotation via map-reduce. Since the corpus is heavily pruned before this stage (only conditions with the relevant ICD codes, only queries that pass the divergence filter), each per-condition candidate pool should not be too large. However, to face the long-context that the candidates may give rise to, we can:

(a) **Map**: split the pool into batches of ∼50 chunks (∼10K tokens). For each batch, the LLM extracts facts relevant to the query, cites which chunk(s) each fact came from, and assigns a facet label to each citation.

(b) **Reduce**: a merge pass synthesizes across batches, deduplicates facets, and produces the final gold answer with the complete (facet → chunk-ids) mapping.

Phase 4: Evaluation

- **Aspect recall**: fraction of annotated facets covered by selected k chunks (strict chunk-id matching).
  $\text{AR}(S) = |\{f \in F : S \cap G_f \neq \emptyset\}|/|F|$, where G_f is the set of gold chunk-ids tagged with facet f.
- **Fac score**: facility-location objective value $g(S) = \sum_{i \in D} \max_{j \in S} w_{ij}$
- **Jaccard**: overlap between method's selection and top-k baseline

- **DocRec / FactRec**: gold document/fact fraction recovered

Methods to compare: top-k (baseline), MMR, gMMR, facility-location

5 Embeddings: considerations

5.1 Contextual Embeddings

A raw chunk from Brief Hospital Course might read:

> *"Pt had L-sided weakness, slurred speech, presented within 3h window. tPA administered. CT head negative for hemorrhage."*

Without context, embedding this chunk places it in a generic "neurological emergency" region. A query about "stroke symptoms" may not rank it highly, because the semantics is different.

Contextual Retrieval [1] Prepend contextual information to each chunk before embedding. Reported: 49% reduction in retrieval failure rate (combined with BM25 hybrid).
The original method uses an LLM to generate the prefix from the surrounding document, since document structure is unknown in the general case. MIMIC discharge notes, however, come with structured metadata (ICD codes, demographics, admission details) already parsed in relational tables, so we replace the LLM call with a deterministic template.

References

[1] Anthropic. Contextual retrieval, 2024. Blog post.

Appendix

A Available Analytics from `mimic-code`

The official `mimic-code` repository provides SQL concepts, cohort definitions, and mapping tables that we can use.

Resource	Location	Role in Our Pipeline
Charlson Comorbidity Index	`mimic-iv/concepts/\protect\ penalty\z@comorbidity/ charlson.sql`	Maps ICD-9/10 codes to 16 clinical categories (MI, CHF, stroke, COPD, diabetes, renal, cancer…). Primary tool for **condition selection** and **comorbidity modifier axes**.
CCS hierarchical grouping	`mimic-iii/concepts_ postgres/\protect\penalty\ z@diagnosis/ccs_dx.sql` + `ccs_multi_dx.csv.gz`	4-level hierarchical ICD-9 → clinical category mapping. Useful for building the macrocategory index in hierarchical retrieval (Phase 2).
Concept maps	`mimic-iv/concepts/\protect\ penalty\z@concept_map/`	ICD → LOINC/OMOP/SNOMED CSVs. Cross-referencing structured data when building contextual embedding prefixes.
Age / demographics	`mimic-iv/concepts/\protect\ penalty\z@demographics/age. sql`	Age cohort extraction. Needed for demographic modifier axes.
Severity scores	`mimic-iv/concepts/\protect\ penalty\z@score/` (SOFA, SAPS-II, LODS, APS-III, OASIS)	Patient acuity signals. Could serve as additional metadata for contextual embeddings or as a proxy for clinical complexity.